

CS253 Assignment 1

Memory-Efficient Versioned File Indexer

Velpula Jashwanth
241157

March 9, 2026

1 Introduction

This assignment implements a memory-efficient versioned file indexer in C++. The system processes large text files using a fixed-size buffer and constructs a word-level index mapping each unique word to its frequency.

The goal of the system is to demonstrate efficient memory usage, modular object-oriented design, and support for multiple analytical queries such as word count, top-K frequent words, and frequency differences between file versions.

2 Design and Approach

The program reads the input file incrementally using a fixed-size buffer instead of loading the entire file into memory. This ensures that memory usage remains independent of the file size.

The design consists of multiple classes with clearly defined responsibilities:

- Buffered file reading for memory-efficient file processing.
- Tokenization for extracting words from text.
- Versioned indexing for storing word frequencies.
- Query processing for answering analytical queries.

The indexing process works as follows:

1. The file is read in chunks using a fixed-size buffer.
2. Each chunk is tokenized into words.
3. Words are converted to lowercase to ensure case-insensitive matching.
4. Word frequencies are stored in a hash map.
5. Each indexed file is stored as a separate version.

3 Class Descriptions

3.1 BufferedReader

This class is responsible for reading the file incrementally using a fixed-size buffer. It ensures that only small portions of the file are loaded into memory at any time.

Key Responsibilities:

- Open input files
- Read file contents chunk-by-chunk
- Maintain a fixed buffer size between 256KB and 1024KB

3.2 Tokenizer

The Tokenizer class processes chunks of text and extracts words from them.

Words are defined as sequences of alphanumeric characters. The tokenizer also handles cases where words may be split across buffer boundaries.

Key Responsibilities:

- Extract words from text
- Convert words to lowercase
- Handle partial tokens between buffers

3.3 VersionedIndex

This class stores word frequency mappings for different file versions.

Each version corresponds to a separate dataset and maintains its own word frequency map.

Key Responsibilities:

- Build word frequency indexes
- Store multiple file versions
- Provide access to frequency data for queries

3.4 Query Classes

Query processing is implemented using an abstract base class called `Query`. Different query types inherit from this class.

Derived classes include:

- `WordQuery` – retrieves the frequency of a word
- `TopQuery` – retrieves the top-K most frequent words
- `DiffQuery` – compares word frequency between two versions

This design demonstrates runtime polymorphism through virtual functions.

4 C++ Features Used

The implementation demonstrates several important C++ programming concepts:

- Object-Oriented Programming
- Inheritance and abstract base classes
- Runtime polymorphism using virtual functions
- Templates for generic utilities
- Exception handling using try-catch blocks
- Standard Template Library (STL) containers

5 Supported Queries

The program supports three query types:

5.1 Word Count Query

Returns the frequency of a specified word in a given version.

Example command:

```
.\analyzer.exe --file dataset.txt --version v1 --buffer 512 --query word --  
--word error
```

5.2 Top-K Query

Returns the top K most frequent words in a version.

Example command:

```
.\analyzer.exe --file dataset.txt --version v1 --buffer 512 --query top --  
top 10
```

5.3 Difference Query

Computes the difference in frequency of a word between two versions.

Example command:

```
.\analyzer.exe --file1 d1.txt --version1 v1 --file2 d2.txt --version2 v2 --  
buffer 512 --query diff --word error
```

6 Assumptions and Observations

- Words are considered as sequences of alphanumeric characters.
- Word matching is case-insensitive.
- Memory usage grows only with the number of unique words.
- Files may be arbitrarily large since they are processed incrementally.
- Buffer size remains constant throughout execution.

7 Conclusion

The implemented system efficiently processes large text files using a fixed-size buffer and builds versioned word indexes. The modular object-oriented design ensures clear separation of responsibilities between classes.

The program successfully demonstrates memory-efficient file processing, runtime polymorphism, templates, and exception handling in C++.